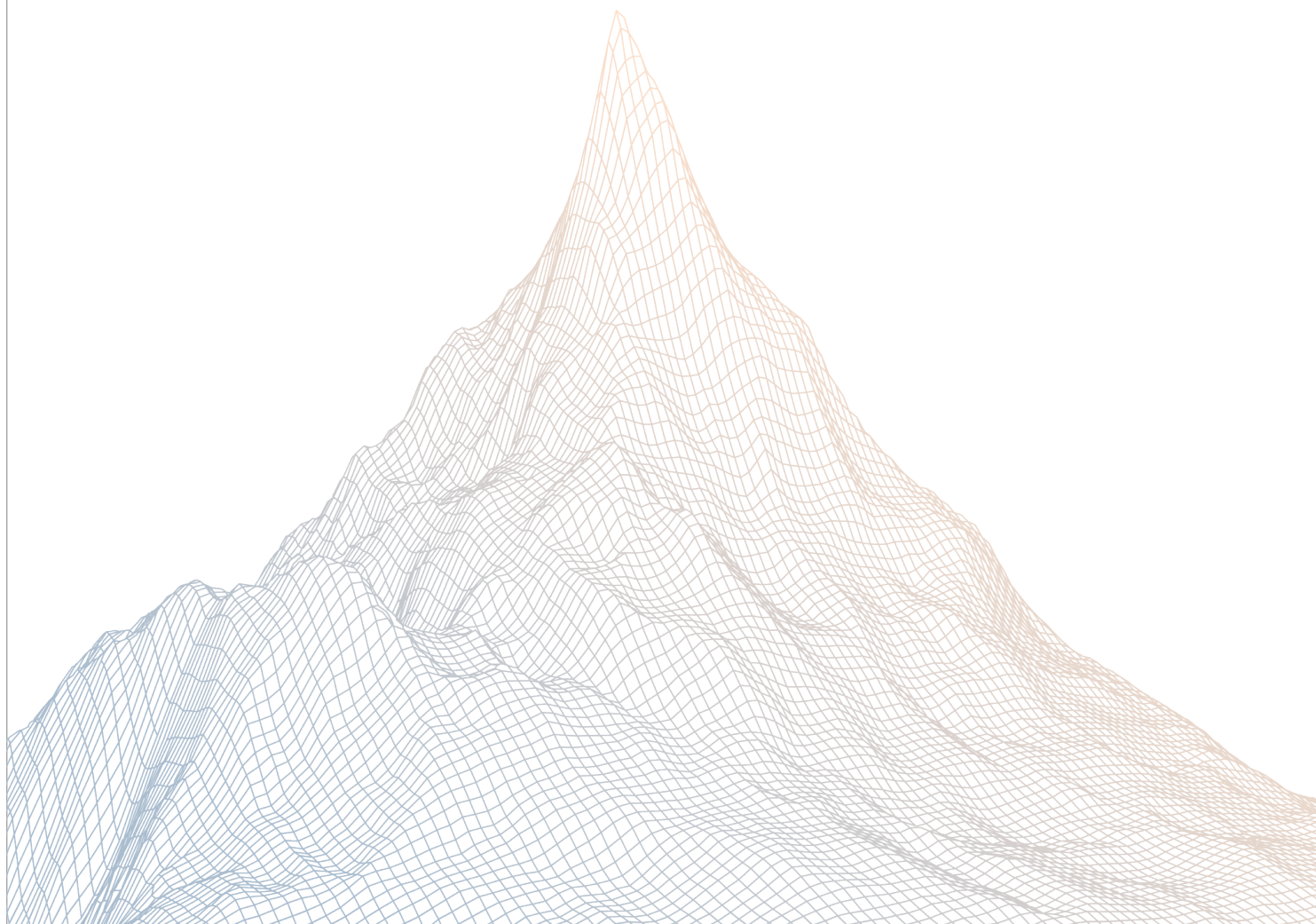


Interfold

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

September 5th to 8th, 2026

AUDITED BY:

Mario Ponder
said

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Interfold	4
2.2	Scope	5
2.3	Audit Timeline	7
2.4	Issues Found	7
3	Findings Summary	7
4	Findings	9
4.1	Medium Risk	10
4.2	Low Risk	15
4.3	Informational	22

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Interfold

The Interfold enables applications that coordinate computation across independent parties without exposing their private inputs. Instead of pooling sensitive data or delegating execution authority to a centralized operator, the protocol executes encrypted workflows across a distributed network of ciphernodes.

2.2 Scope

The engagement involved a review of the following targets:

Target	interfold
Repository	https://github.com/theinterfold/interfold
Commit Hash	723bed2eeb93beb20900a444bf7c78aa2d8cff16
Files	Diff from 10ac245af1ecb8873a332bb188dad2972903ae18 packages/interfold-contracts/contracts/Interfold.sol packages/interfold-contracts/contracts/interfaces/ICiphernodeRegistry.sol packages/interfold-contracts/contracts/interfaces/IDataAvailabilityVerifier.sol packages/interfold-contracts/contracts/interfaces/IE3.sol packages/interfold-contracts/contracts/interfaces/IInterfold.sol packages/interfold-contracts/contracts/interfaces/external/IAvailBridge.sol packages/interfold-contracts/contracts/lib/InterfoldLifecycle.sol packages/interfold-contracts/contracts/lib/InterfoldPricing.sol packages/interfold-contracts/contracts/lib/RegistrySortitionLib.sol packages/interfold-contracts/contracts/registry/CiphernodeRegistryOwnable.sol ../contracts/verifiers/AvailVectorXDataAvailabilityVerifier.sol crates/support/contracts/ImageID.sol examples/CRISP/packages/crisp-contracts/contracts/CRISPProgram.sol templates/default/contracts/MyProgram.sol

Target	Interfold Mitigation Review
Repository	https://github.com/theinterfold/interfold
Commit Hash	95c38d7060cf4187c7795ac94d0c57c9fc16e464
Files	Diff from 10ac245af1ecb8873a332bb188dad2972903ae18 packages/interfold-contracts/contracts/Interfold.sol packages/interfold- contracts/contracts/interfaces/ICiphernodeRegistry.sol packages/interfold- contracts/contracts/interfaces/IDataAvailabilityVerifier.sol packages/interfold-contracts/contracts/interfaces/IE3.sol packages/interfold- contracts/contracts/interfaces/IInterfold.sol packages/interfold- contracts/contracts/interfaces/external/IAvailBridge.sol packages/interfold- contracts/contracts/lib/InterfoldLifecycle.sol packages/interfold-contracts/contracts/lib/InterfoldPricing.sol packages/interfold- contracts/contracts/lib/RegistrySortitionLib.sol packages/interfold- contracts/contracts/registry/CiphernodeRegistryOwnable.sol ../.. /contracts/verifiers/AvailVectorXDataAvailabilityVerifier.sol crates/support/contracts/ImageID.sol examples/CRISP/packages/crisp- contracts/contracts/CRISPProgram.sol templates/default/contracts/MyProgram.sol

2.3 Audit Timeline

September 5, 2026	Audit start
September 8, 2026	Audit end
September 11, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	2
Low Risk	5
Informational	10
Total Issues	17

3

Findings Summary

ID	Description	Status
M-1	Credited rewards bypass later expulsion holds and forfeiture	Resolved
M-2	A committee operator can release its bond and withdraw all collateral inside the accusation window, escaping slashing	Resolved
L-1	Application verification can overwrite terminal E3 failure	Resolved
L-2	VectorX pointer rotation permanently bricks in-flight rounds	Resolved
L-3	The randomness breaker is registry-wide and permissionlessly trippable, and VRF admission reserves nothing	Resolved
L-4	The Avail adapter mints an availability receipt for a zero content hash	Resolved
L-5	A committee member can front-run a committee-affecting slash with markE3Failed to lock in requester-pays attribution	Resolved
I-1	Large payloads and repeated key events increase processing costs	Resolved
I-2	Fee and voting-power calculations use floor division	Resolved
I-3	Data availability timeouts allocate losses by lifecycle stage	Resolved
I-4	Tree capacity is one leaf below the advertised maximum	Resolved
I-5	Owner can initialize CRISP round state for an E3 assigned to another program	Resolved
I-6	The tracked CRISP image ID copy retains an obsolete value	Resolved
I-7	Any account can write to another voter's input slot, with no on-chain sender or sibling-ordering binding	Acknowledged

ID	Description	Status
I-8	Committee key publication after the input window flips failure attribution to the requester	Resolved
I-9	A committed input with no availability receipt bricks the round, billed to the requester	Resolved
I-10	An E3 program not implementing IE3ProgramDataAvailability bricks its own output publication	Resolved

4

Findings

4.1 Medium Risk

A total of 2 medium risk findings were identified.

[M-1] Credited rewards bypass later expulsion holds and forfeiture

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [../contracts/lib/InterfoldPricing.sol#L72-L83](#)
- [../contracts/lib/InterfoldPricing.sol#L311-L328](#)
- [../contracts/Interfold.sol#L1164-L1192](#)
- [../contracts/slashing/SlashingManager.sol#L534-L608](#)
- [../contracts/E3RefundManager.sol#L764-L815](#)
- [../contracts/E3RefundManager.sol#L1180-L1188](#)
- [../contracts/E3RefundManager.sol#L1277-L1286](#)

Description:

Successful E3 settlement checks each operator's pending expulsion status when allocating its reward. If no proposal is pending, the allocation is credited to the operator's frozen reward recipient in `_pendingRewards[e3Id][recipient]`.

A valid expelling proposal can open after completion. However, opening or executing that proposal does not recover the credited balance. Both `claimReward` and `claimRewards` pay through the following code without checking the operator's updated entitlement:

```
amount = pendingRewards[e3Id][account];
if (amount == 0) return 0;
pendingRewards[e3Id][account] = 0;
IERC20 token = feeTokens[e3Id];
_transferExact(token, account, amount);
```

Thus, `complete` → `proposal` → `claim` permits withdrawal during the required hold. Alternatively, `complete` → `proposal` → `execute expulsion` → `claim` permits withdrawal after

the allocation should have been forfeited and reallocated. Resolution redistributes rewards still held against the operator, but cannot reach amounts already moved into recipient-based claim balances.

The frozen reward recipient can perform these withdrawals. The permissionless Lane A proposal path requires an enabled committee-expelling policy, valid signed quorum evidence from active non-accused members, an unused evidence key, and compliance with signature and submission deadlines. Completion preserves the committee records needed for these checks. Deferred execution permits the pending-proposal sequence; atomic execution still permits withdrawal afterward.

The same accounting gap affects two additional paths:

- Clearing the last pending proposal moves held success rewards into `_pendingHeldSuccessClaims`. A subsequent valid proposal does not prevent `claimHeldSuccessReward` from paying. On Lane A, that proposal requires another unused, enabled proof type; changing evidence bytes alone is insufficient.
- A node share credited from another operator's non-expelling ticket penalty remains payable through `claimSlashedFunds` despite a later pending or executed expulsion against its beneficiary.

This violates the order-independent reward-eligibility requirement in `agent/INVARIANTS.md`, lines 311-316. Unresolved proposals must hold the accused operator's unclaimed allocations, and execution must reallocate them. Only rewards withdrawn before proposal opening remain final.

The impact is bounded by existing funded allocations. The recipient can retain rewards that should be held or redistributed, reducing eligible recipients' allocations after expulsion. Each withdrawal consumes its claim balance, and withdrawing rewards does not bypass the separate collateral-slashing operation.

Recommendations:

It is recommended to preserve each allocation's E3, operator, frozen recipient, token, and amount until withdrawal. Check both pending expulsion and exclusion through the E3's snapshotted entitlement authority. The existing `rewardDisposition` function reports only pending status and is insufficient by itself.

Hold affected allocations while any expelling proposal remains pending. Release them only when all such proposals have cleared and the operator is not excluded. On execution, reallocate forfeited amounts through resolution or permissionless settlement without requiring the accused recipient to request withdrawal. Preserve independent claims for eligible allocations, including those sharing a recipient address.

Apply these rules to credited, released, and redistributed rewards. Preserve the finality of withdrawals made before proposal opening.

Migrate existing recipient-only balances with verified operator attribution and preserve proxy storage layout, token liabilities, and E3-specific dependencies. Prevent legacy and

migrated claim paths from paying the same allocation twice. A deployment with zero active E3s can still have outstanding reward claims.

Interfold: Resolved with [PR-1928](#) by checking operator eligibility at withdrawal and preserving each penalty's original target through redistribution.

Zenith: Verified.

[M-2] A committee operator can release its bond and withdraw all collateral inside the accusation window, escaping slashing

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [../contracts/registry/CiphernodeRegistryOwnable.sol#L777-L798](#)
- [../contracts/lib/BondingSlashingLib.sol#L41-L49](#)
- [../contracts/slashing/SlashingManager.sol#L375-L382](#)

Description:

releaseCommittee is public and gated only on the round being Complete or Failed:

```
function releaseCommittee(uint256 e3Id) public {
    Committee storage c = committees[e3Id];
    require(c.stage == ... Requested || c.stage == ... Finalized,
        CommitteeNotFinalized());
    ...
    IInterfold.E3Stage stage = _interfoldFor(e3Id).getE3Stage(e3Id);
    if (stage != IInterfold.E3Stage.Complete
        && stage != IInterfold.E3Stage.Failed) revert E3NotTerminal(e3Id);
    c.obligationsReleased = true; // no accusation-window gate, unlike closeE3
    RegistrySortitionLib.failRequestedCommittee(c, e3Id);
    _releaseCommitteeObligations(e3Id,
        c); // clears the committee-obligation hold on exit claims
}
```

releaseCommittee removes the completed or failed round's hold on an operator's collateral. However, it does not wait until the deadline for submitting accusations has passed, even though closeE3 enforces that deadline. This creates a period in which the operator may withdraw collateral that should still be available for slashing.

The operator must still satisfy the other withdrawal requirements: the exit delay must have passed, no other committee obligation or open slash proposal may block withdrawal, and the caller must own the collateral (BondingSlashingLib.sol:41-49). An active committee member can call markE3Failed during the grace period after a real timeout, it cannot arbitrarily fail a healthy round. Once the round is terminal, the operator can release its

committee obligation and withdraw when the remaining conditions are satisfied. A valid accusation submitted afterward can then result in a slash against zero remaining collateral.

The accusation deadline is calculated from the round's request-time worst-case lifecycle, plus one day—not from when the round actually completes or fails (`SlashingManager.sol:351-364`). On mainnet, both the exit delay and maximum lifecycle are 30 days. Consequently, an early-queued exit can mature before accusations close for a round configured near that maximum. Shorter rounds generally leave insufficient time for this sequence.

For example, consider this illustrative timeline:

1. **Day 0 — Request:** A round is configured with a worst-case lifecycle ending on day 30. Its accusation deadline is therefore day 31.
2. **Day 0.1 — Queue exit:** After joining the committee, an operator queues its entire 1,000 FOLD stake for withdrawal. The 30-day exit delay makes it claimable on day 30.1.
3. **Day 1 — Actual failure:** A timeout occurs and the round is marked failed. Its accusation deadline remains day 31.
4. **Release the obligation:** The operator calls `releaseCommittee`. The failed round no longer blocks its withdrawal.
5. **Day 30.1 — Withdraw:** Assuming no other committee obligations or open slash locks, the owner withdraws all 1,000 FOLD.
6. **Day 30.5 — Accuse:** A valid accusation is submitted before day 31. The resulting slash finds no remaining collateral.

The missing protection is a collateral hold through the accusation deadline, even when the round ends earlier.

Recommendations:

Gate `releaseCommittee` on the slashing manager's accusation window having closed, the same condition `closeE3` enforces, or block exit claims while `slashSubmissionDeadline` for any of the operator's committees has not passed.

Interfold: Resolved with [PR-1928](#). `releaseCommittee` blocks a finalized committee until `accusationSubmissionDeadline` passes.

Zenith: Verified.

4.2 Low Risk

A total of 5 low risk findings were identified.

[L-1] Application verification can overwrite terminal E3 failure

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../contracts/lib/InterfoldLifecycle.sol#L317-L330](#)
- [../contracts/Interfold.sol#L896-L906](#)

Description:

publishCiphertext checks the E3 stage and committee viability before verification. `_isValidCiphertextHash` then calls `IE3Program.verify`, whose interface permits state changes. After verification succeeds, publication writes `CiphertextReady` without rechecking the stage or viability:

```
if (
    !_isValidCiphertextHash(
        e3,
        e3Id,
        contentHash,
        ciphertextCommitment,
        outputReference.computeProof
    )
) revert IInterfold.InvalidOutput(bytes(""));

e3.ciphertextOutput = contentHash;
e3.ciphertextCommitment = ciphertextCommitment;
stages[e3Id] = IInterfold.E3Stage.CiphertextReady;
deadlines[e3Id].decryptionDeadline = block.timestamp + decryptionWindow;
```

An owner-registered application whose verification callback can execute an expelling slash can trigger this conditional sequence:

1. Publication begins in `KeyPublished`, within its permitted time window, with a viable committee and valid availability and protocol proofs.
2. The callback executes an existing, mature slash proposal that reduces active membership below `H`. The proposal must satisfy the applicable evidence, quorum, and appeal requirements.
3. The authorized slashing manager calls `onE3Failed`, which is outside the publication reentrancy guard. It records `Failed`, stores `InsufficientCommitteeMembers`, and decrements `activeE3Count`.
4. The application returns `true`. Publication overwrites `Failed` without restoring the counter or clearing the failure reason.

A subsequent timeout attempts another decrement. At zero, it reverts; otherwise, it can consume another active E3's contribution and later block that request's completion or failure processing.

The callback can also invoke `processE3Failure` and `releaseCommittee` while the E3 is temporarily `Failed`. Those operations are authorized at that point, but their effects persist after publication restores a nonterminal stage. This creates inconsistent lifecycle, settlement, and committee-obligation state; it does not independently establish refund theft. Plaintext verification still rejects insufficient committee viability.

This violates the "Committee viability loss is atomic" invariant in `agent/INVARIANTS.md`. The change removed the previous post-verification stage check.

Recommendations:

It is recommended to retain the initial checks and, immediately after application verification succeeds, read the current E3 stage from storage and recheck request-time committee viability. Require `KeyPublished` and a viable committee before storing the output or advancing the stage. A failed check must revert all nested failure, settlement, and committee-release effects; the legitimate slash can still execute separately.

If stateful application verification is unnecessary, enforcing static verification through an interface change is an alternative that requires compatibility review.

Interfold: Resolved with [PR-1926](#) by rechecking the E3 stage and request-time committee viability after application verification, reverting nested effects if either check fails.

Zenith: Verified.

[L-2] VectorX pointer rotation permanently bricks in-flight rounds

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../contracts/verifiers/AvailVectorXDataAvailabilityVerifier.sol#L46-L48](#)

Description:

verifyDataAvailability re-checks the bridge pointer on every call and reverts:

```
// AvailVectorXDataAvailabilityVerifier.verifyDataAvailability
▷ if (address(bridge.vectorx()) ≠ address(vectorx))
    revert InvalidVectorX();
// both `bridge` and `vectorx` are immutable; there is no re-point path
```

The adapter's VectorX address is fixed at deployment. If Avail governance changes the bridge's VectorX pointer, the addresses stop matching and receipt verification reverts with `InvalidVectorX`. Rounds that still need to finalize inputs or publish their ciphertext output are then blocked.

An in-flight round has no way to switch to a replacement adapter or extend its compute deadline. If the mismatch lasts through that deadline, the round loses its opportunity to complete. Restoring the original pointer afterward is too late.

Recommendations:

Provide a governed, timelocked path to re-point the DA verifier for in-flight rounds or a bounded deadline extension, and document the runbook for detecting a rotation and migrating affected rounds.

Interfold: Resolved with [PR-1928](#). The limitation is documented.

Zenith: Verified.

[L-3] The randomness breaker is registry-wide and permissionlessly trippable, and VRF admission reserves nothing

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../contracts/lib/RegistrySortitionLib.sol#L353-L374](#)

Description:

When one round's randomness request passes its frozen deadline unfulfilled, the breaker zeroes the single provider address held in registry storage:

```
function _tripRandomnessCircuitBreaker(uint256 e3Id) private {
    RandomnessStorage storage state = _randomnessStorage();
    RandomnessRequest storage request = state.requests[e3Id];
    if (address(state.provider) == address(0) || ...
        || block.timestamp <= request.randomnessDeadline) return;
    (bool ready, , ) = _providerState(request, e3Id);
    if (ready) return;
    > state.provider = IRandomnessProvider(address(0)); // registry-GLOBAL
    provider, not per-E3
    emit RandomnessCircuitBreakerTripped(e3Id, request.requestId,
        failedProvider);
}
```

It is reachable through the permissionless finalizeCommittee timeout branch, markE3Failed, and the requester's cancelE3; every subsequent Interfold.request then reverts ZeroAddress. Recovery needs requests paused and unreleasedCommitteeCount = 0, so a pre-armed long-lived round can hold it off for up to its 30-day windows. Separately, ChainlinkVrfRandomnessProvider.requestRandomness checks only the current subscription balance and reserves nothing for in-flight draws, so a one-block burst passes the same check, underfunded requests stay pending 24 hours but the frozen one-hour deadline makes a late fulfillment unusable, which is how an attacker manufactures the non-fulfillment that trips the breaker. Each round the attacker uses prepays its own quote, the marginal lever is the flat randomness fee, and a request funded to deplete the subscription continues into normal chargeable work.

Recommendations:

Scope the breaker to a failure count or make it advisory and let governance re-point without draining every committee, track outstanding VRF requests and require $\text{availableBalance} \geq \text{minimum} + \text{pending} * \text{maxRequestCost}$, or cap concurrent unfulfilled draws.

Interfold: Resolved with [PR-1926](#).

Zenith: Verified.

[L-4] The Avail adapter mints an availability receipt for a zero content hash

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../contracts/verifiers/AvailVectorXDataAvailabilityVerifier.sol#L41-L79](#)
- [CRISPPProgram.sol#L633-L666](#)

Description:

Avail pads its submitted-data Merkle tree with `H256::zero()` leaves, and the bridge hashes the caller-supplied pre-image, so a padding leaf is a provable leaf whose pre-image is `bytes32(0)` with a public Merkle path. The adapter's only content binding has no zero guard:

```
// AvailVectorXDataAvailabilityVerifier.verifyDataAvailability
▷ if (input.leaf ≠ expectedContentHash)
    revert ContentHashMismatch(expectedContentHash, input.leaf);
// expectedContentHash = 0 is never rejected; a zero padding leaf
// satisfies this equality
if (!bridge.verifyBlobLeaf(input)) revert InvalidAvailabilityProof();
```

so a zero expected hash passes and the adapter returns a receipt for bytes never published. `InterfoldLifecycle` is protected by a separate zero-hash check; `CRISPPProgram.finalizeInput` is not, and the hash is not a Noir public input, so nothing on-chain constrains it to be non-zero. Reachable if the trusted signer signs for a zero hash, or by a future consumer of this reusable adapter that omits the guard `Interfold` happens to have.

Recommendations:

Add `if (expectedContentHash == bytes32(0)) revert ...` in the adapter, where every consumer inherits it, and optionally reject a zero `encryptedVoteHash` in `publishInput`.

Interfold: Resolved with [PR-1926](#).

Zenith: Verified.

[L-5] A committee member can front-run a committee-affecting slash with `markE3Failed` to lock in requester-pays attribution

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../contracts/slashing/SlashingManager.sol#L823-L833](#)
- [Interfold.sol#L873-L907](#)

Description:

The failure reason is written once and is irreversible. `_executeSlash` reclassifies a failure only if the round is not already terminal:

```
if (activeCount < thresholdM) {
    IInterfold.E3Stage stage
        = dependencies.interfoldContract.getE3Stage(p.e3Id);
    if (stage != IInterfold.E3Stage.Complete && stage !=
        IInterfold.E3Stage.Failed) {
        dependencies.interfoldContract.onE3Failed(p.e3Id,
            ...); // skipped if markE3Failed already made it Failed
    }
}
```

At `KeyPublished` past the compute deadline an active member may call `markE3Failed` in the grace period, recording `ComputeTimeout`, after which the slash still executes but the reason stays. With the deployed 1000/4000/500 bps allocation this refunds the requester 45% instead of 100%; the remaining 55% is split to nodes (50%) and protocol (5%).

Recommendations:

Let `_executeSlash` overwrite a supplier-favourable reason when an expulsion drops the committee below threshold, or order `markE3Failed` behind pending executable slashes; couple with settlement-finality so it does not conflict with an already-calculated distribution.

Interfold: Resolved with [PR-1928](#) and Resolved with [PR-1926](#)

Zenith: Verified.

4.3 Informational

A total of 10 informational findings were identified.

[I-1] Large payloads and repeated key events increase processing costs

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../contracts/verifiers/AvailVectorXDataAvailabilityVerifier.sol#L41-L79](#)
- [../contracts/Interfold.sol#L449-L484](#)
- [../contracts/lib/RegistrySortitionLib.sol#L22-L97](#)
- [examples/CRISP/packages/crisp-contracts/contracts/CRISPProgram.sol#L633-L656](#)
- [../verifiers/bfv/honk/secure-8192/minimum/DkgAggregatorVerifier.sol#L2532-L2547](#)

Description:

Several publication interfaces accept dynamic data without additional application-level length caps. The VectorX adapter decodes dynamic Merkle-proof arrays before calling the bridge:

```
IAvailBridge.MerkleProofInput memory input = abi.decode(
    proof,
    (IAvailBridge.MerkleProofInput)
);
if (input.leaf != expectedContentHash) {
    revert ContentHashMismatch(expectedContentHash, input.leaf);
}
if (!bridge.verifyBlobLeaf(input)) revert InvalidAvailabilityProof();
```

Plaintext output and several proof parameters similarly lack wrapper-level caps. Large payloads can increase calldata, decoding, verification, storage, and event-processing costs, depending on the path. These are documented operational constraints. Verifier-specific restrictions still apply; for example, the inspected DKG Honk verifier

requires an exact proof length.

Required verification continues to protect input and output acceptance. Direct verification failures and uncaught out-of-gas conditions revert the affected publication call. They do not finalize an input or partially accept a ciphertext. Plaintext verification failures also roll back state written earlier in that call.

Committee-key publication enforces a maximum of 512 KiB per declared candidate and 90 KiB per chunk payload, with canonical chunk counts and lengths. However, it does not record previously emitted chunks or candidates. A committee member whose status is not `None` can repeatedly emit validly formatted chunks while the E3 remains in `KeyPublished`. The maximum of six chunks per candidate does not cap total publication transactions. The function also has no independent deadline check.

Repeated publication does not append on-chain candidate state, but observers still process the resulting logs. Existing consumer controls select one candidate per publisher and bound retained assembly state, consistent with the public-key invariant. They do not eliminate repeated log-processing costs.

Transaction senders pay gas, including services acting as relayers. Observers incur separate processing costs. The availability service obtains inclusion proofs and simulates finalization before submission, so the missing contract caps do not establish arbitrary service-funded execution. Simulation itself consumes RPC resources and cannot guarantee later transaction success.

Recommendations:

It is recommended to document the existing size limits and the distinction between bounded candidate size and repeatable event publication. Preserve existing object-byte accounting, candidate isolation, assembly limits, and cleanup controls required by the C-01 public-key invariant.

Introduce additional contract caps only where supported proof formats or protocol configurations establish a justified maximum. Where feasible, check the encoded payload length before expensive decoding or copying.

For relayers, assess admission capacity and funding before issuing availability promises. Configure deadline-aware retries, monitoring, and recovery for accepted work. A gas budget or retry cutoff must not silently discard committed data or remove a pending input from accounting. Review observer queue budgets while preserving key-transport recovery. Base operational gas limits on measured complete transactions.

Interfold: Resolved with [PR-1930](#) by documenting existing payload limits, repeatable key-chunk publication, verification guarantees, and observer processing costs.

Zenith: Verified.

[I-2] Fee and voting-power calculations use floor division

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../contracts/lib/InterfoldPricing.sol#L414-L502](#)
- [../contracts/lib/InterfoldPricing.sol#L224-L308](#)
- [../contracts/lib/InterfoldPricing.sol#L72-L96](#)
- [examples/CRISP/packages/crisp-contracts/contracts/CRISPProgram.sol#L567-L581](#)
- [examples/CRISP/packages/crisp-contracts/contracts/CRISPProgram.sol#L798-L825](#)

Description:

The pricing calculation rounds the weighted billable duration down to whole seconds before multiplying it by the availability price and committee size. It sums the weighted terms before division, avoiding a separate rounding loss for each phase:

```
uint256 durationBps = preComputeBps +
    tc.computeWindow *
    uint256(pc.computeUtilizationBps) +
    tc.decryptionWindow *
    uint256(pc.decryptUtilizationBps);
return durationBps / uint256(BPS_BASE);
```

The subsequent margin calculation also rounds down. Let $B = 10,000$, a be the availability price per node per second, n the quoted committee size, and m the margin in basis points. For a successful quote, comparison with the same formula evaluated using rational arithmetic gives:

$$0 \leq Q_{\mathrm{rational}} - Q_{\mathrm{contract}} < a n \left(1 + \frac{m}{B}\right) + 1$$

This is a per-request precision bound in raw fee-token units. It is not generally below one raw unit or one cent, and it does not establish an economically negligible difference. The rational value is a comparison baseline, not an established minimum charge. The randomness fee is added separately without margin. Pair-count divisions are exact because consecutive integers have an even product.

During successful settlement, the treasury share rounds down by less than one raw unit relative to its rational value. That difference remains in the committee pool. For a nonempty

active committee, the pool is divided among its members, and the remainder is included in the allocation for `slot e3Id % activeCount`. Allocations sum to the entire pool, including when the pool is smaller than the active committee. This remainder distribution does not further reduce the calculated treasury share.

Settlement consumes the recorded service escrow, and claims clear their balances before transferring tokens. Repeated claims cannot recreate the same allocation. Rounding differences can accumulate across paid requests, but each request repeats fixed charges, including the positive randomness fee. No profitable request-splitting strategy or conservation failure is established by the inspected arithmetic.

CRISP separately converts raw voting power into integer ballot units. In ONCHAIN CUSTOM mode, initialization requires `minVotingPower ≥ divisor`, and input eligibility checks raw snapshot power against that minimum. Division alone therefore cannot reduce an eligible slot's allowance to zero. This guarantee concerns the allowance, not the submitted vote or tally. ONCHAIN CONSTANT mode uses explicit nonzero credits; Merkle census modes obtain their bound through the census proof.

These calculations and guards predate the reviewed change. They are consistent with the raw-unit pricing and exact-custody accounting invariants. This is an informational observation about precision and allocation policy.

Recommendations:

It is recommended to consolidate the documented units, rounding directions, per-request precision bound, and remainder-allocation policy. Preserve exact pool allocation, claim consumption, and the CUSTOM eligibility floor.

If greater quote precision becomes a requirement, retain fractional weighted duration through the monetary calculation and choose the final rounding direction explicitly. Treat this as a pricing-policy change rather than required security remediation. Voting allowances should not be rounded upward as a generic remedy.

Review existing coverage and add cases where needed for fractional weighted durations, small committee pools, duplicate claim IDs, and eligibility boundaries.

Interfold: Resolved with [PR-1930](#) by documenting the existing fee rounding, raw-unit precision limits, and committee remainder-allocation policy without changing the arithmetic.

Zenith: Verified.

[I-3] Data availability timeouts allocate losses by lifecycle stage

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../contracts/lib/InterfoldLifecycle.sol#L310-L330](#)
- [../contracts/lib/InterfoldLifecycle.sol#L535-L544](#)
- [../contracts/lib/FailurePayerLib.sol#L13-L35](#)
- [../contracts/E3RefundManager.sol#L315-L342](#)
- [crates/evm/src/data_availability/actor.rs#L476-L509](#)

Description:

Data availability failures can produce different financial outcomes depending on whether they prevent aggregate publication or subsequent ciphertext retrieval. Settlement follows the lifecycle stage and timeout reason; it does not separately attribute an outage to external infrastructure.

Before aggregate publication is accepted, an E3 remains in `KeyPublished`. Pending input availability receipts or a missing aggregate availability receipt can prevent this transition. If the compute deadline expires, `ComputeTimeout` is requester-paid. The service escrow funds the committee's snapshotted allocation for completed stages, subject to recipient eligibility, with the remaining amounts distributed under the refund policy. This allocation is a scheduled fee, rather than reimbursement of measured operating costs.

Once aggregate publication passes verification, the contract enters `CiphertextReady` and starts the decryption deadline. This occurs before individual nodes retrieve the ciphertext. The retrieval actor checks the referenced content hash and retries failures, but emits the local event that enables decryption only after retrieval succeeds.

If retrieval failures prevent sufficient decryption participation before the deadline, the E3 can reach `DecryptionTimeout`. This reason is `ciphernode-paid`, for which the refund manager returns:

```
if (getFailurePayer(reason) == FailurePayer.Ciphernodes) {  
    return (0, originalPayment, 0);  
}
```

Consequently, the committee receives no compensation from the service escrow, including

for completed DKG work, while the requester receives that escrow in full. The separately charged randomness fee is excluded. This timeout does not itself slash collateral or reclaim previously distributed rewards; distributions from separate slashing proceeds remain distinct.

The source configuration provides a seven-day compute window, a six-hour decryption window, and a three-hour input-finalization tail. Already committed inputs can be finalized through the compute deadline. These periods provide recovery time, but do not establish outage frequency or guarantee timely retrieval.

This behavior follows the failure-payout invariant in `agent/INVARIANTS.md`, and ciphertext retrieval is part of the committee's assigned duties. No requirement to compensate completed work during infrastructure outages has been established. This is therefore an informational integration observation.

Recommendations:

It is recommended to extend the existing payout and operator documentation with these data availability scenarios, including when the decryption deadline starts and which completed-stage fees remain payable after each timeout.

Monitor retrieval failures and remaining deadline time. Consider redundant RPC access to the same Avail network to reduce endpoint-specific failures, while preserving receipt verification and exact content-hash checks.

If infrastructure failures should receive different settlement treatment, define this as an explicit policy change. Such a policy needs credible failure attribution and a funded allocation that preserves accounting invariants. A node's reported RPC error alone does not establish a network outage, and full requester refunds plus committee compensation require funding beyond the same service escrow. Any change should preserve the policy snapshots of existing requests.

Interfold: Resolved with [PR-1930](#) by documenting the existing lifecycle-based data availability timeout payouts and funding requirements for alternative settlement.

Zenith: Verified.

[I-4] Tree capacity is one leaf below the advertised maximum

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../../contracts/registry/CiphernodeRegistryOwnable.sol#L100-L114](#)
- [../../contracts/registry/CiphernodeRegistryOwnable.sol#L544-L570](#)
- [examples/CRISP/packages/crisp-contracts/contracts/CRISPProgram.sol#L990-L1000](#)
- [templates/default/contracts/MyProgram.sol#L89-L92](#)
- [compiler-inputs/...10ef2e81b222675b5fc87c847a413a4af880b73da2f8.json#L281-L283](#)

Description:

The registry exposes `MAX_CIPHERNODE_LEAVES` as `1 << TREE_DEPTH`, advertising 1,048,576 leaves at depth 20. However, the pinned `@zk-kit/lazy-imt.sol` version `2.0.0-beta.12` permits only 1,048,575 leaves. CRISP and the default program template use the same dependency and initialization depth.

The dependency initializes its limit as the highest theoretical index, then uses that value as an exclusive insertion bound:

```
// In _init:
self.maxIndex = uint40((1 << depth) - 1);

// In _insert:
uint40 index = self.numberOfLeaves;
require(index < self.maxIndex, "LazyIMT: tree is full");
```

Successful insertions therefore occupy indices 0 through 1,048,574. The 1,048,576th insertion attempts index 1,048,575 and reverts. In the registry, this attempt passes the host's capacity check but fails inside the dependency, so the advertised maximum and custom exhaustion guard do not reflect the effective limit.

Registry exhaustion requires that no reusable index remains. Removing a node makes its index available for `_update`, allowing subsequent registration without another append. A rejected append leaves membership, counters, and the existing root unchanged.

In CRISP, capacity counts appended input entries, including revotes and masks, rather than

distinct voters. The additional insertion fails during input commitment, before its status and pending count are recorded. Transaction rollback also restores the duplicate-leaf flag. Reaching capacity does not prevent finalization of previously committed inputs.

Recommendations:

It is recommended to set `MAX_CIPHERNODE_LEAVES` to `(uint256(1) << TREE_DEPTH) - 1`, retaining the existing comparison and free-index reuse. Update the capacity documentation and the existing test assertion to reflect 1,048,575 leaves. This alignment requires no tree-state migration or reinitialization.

If the full advertised capacity is required, separately validate a corrected dependency across insertion, update, storage indexing, root calculation, and proof generation. Avoid treating a replacement of `<` with `≤` as universally safe. Any migration requirements depend on the chosen correction; changing `_init` alone does not update limits already stored in existing trees.

Interfold: Resolved with [PR-1927](#) by aligning the registry's advertised capacity and exhaustion guard with LazyIMT's 1,048,575-leaf limit.

Zenith: Verified.

[I-5] Owner can initialize CRISP round state for an E3 assigned to another program

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [examples/CRISP/packages/crisp-contracts/contracts/CRISPProgram.sol#L442-L485](#)
- [examples/CRISP/packages/crisp-contracts/contracts/CRISPProgram.sol#L699-L725](#)
- [../contracts/lib/InterfoldLifecycle.sol#L408-L435](#)
- [examples/CRISP/server/src/server/indexer.rs#L59-L82](#)
- [examples/CRISP/server/src/server/data_availability.rs#L513-L565](#)
- [examples/CRISP/server/src/server/data_availability.rs#L1171-L1219](#)

Description:

CRISPProgram.validate() permits the CRISP owner to initialize round configuration and an input tree without checking that Interfold assigned the E3 to this program:

```
if (msg.sender != address(interfold) && msg.sender != owner())
    revert CallerNotAuthorized();
if (e3Data[e3Id].paramsHash != bytes32(0)) revert E3AlreadyInitialized();

_initRound(e3Id, customParams);
_validateInputTiming(e3Id);

e3Data[e3Id].paramsHash = keccak256(e3ProgramParams);
e3Data[e3Id].votes._init(TREE_DEPTH);
```

With the production finalization reserve, timing validation retrieves the existing E3 and checks its configured windows. An E3 assigned to another program can satisfy these checks. The owner can therefore create separate CRISP state for that identifier.

The shared input guards also omit the selected-program comparison. Inputs can enter this separate tree if the E3 is in KeyPublished, the commitment window remains open, and the inputs satisfy CRISP's eligibility, ballot-proof, committee-key, and availability-attestation requirements.

Interfold continues dispatching output verification to the E3's recorded program, preserving

the program-snapshot invariant in `agent/INVARIANTS.md`. Initializing CRISP state does not make its separate input tree or pending-input count authoritative for that program.

The CRISP indexer ignores requests assigned to other programs, limiting exposure through ordinary round discovery. However, direct voting submissions reach an availability service that relies on CRISP's input validation without independently checking program identity. Following owner initialization, valid submissions can cause signing and publication work for inputs outside the selected program's normal computation flow. On mainnet, the service issues an attestation and starts Avail publication only after the input commitment is confirmed in finalized Ethereum state.

This is a pre-existing, owner-dependent misconfiguration risk. Initialization alone does not incur those publication costs. The demonstrated consequence is conditional wasted work; the selected program and its authoritative tally remain unchanged.

Recommendations:

It is recommended to require `address(e3.e3Program) == address(this)` before initializing CRISP round state, independently of the finalization-reserve setting. Apply the same comparison in `_keyPublishedE3()` as defense in depth for input validation, commitment, and finalization.

Interfold stores the provisional E3 and its selected program before calling `validate()`, so this identity check supports normal initialization. Do not require `KeyPublished` during initialization.

If direct owner initialization is needed only for tests, restrict production initialization to Interfold and move owner-driven setup into a dedicated test harness.

Interfold: Resolved with [PR-1927](#) by checking the E3's selected program before CRISP initialization and input validation, commitment, and finalization.

Zenith: Verified.

[I-6] The tracked CRISP image ID copy retains an obsolete value

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [examples/CRISP/.interfold/generated/contracts/ImageID.sol#L21-L23](#)
- [crates/support/contracts/ImageID.sol#L21-L23](#)
- [examples/CRISP/packages/crisp-contracts/deploy/crisp.ts#L22-L29](#)
- [../scripts/upgrade/validateSecureCrisp.ts#L447-L460](#)

Description:

PR 1915 updates the canonical CRISP guest image ID but leaves the tracked generated copy in the example directory unchanged. The example copy retains the canonical file's pre-PR value:

```
// crates/support/contracts/ImageID.sol
bytes32 public constant PROGRAM_ID
= bytes32
  (0x9d3e21fd7cc08e629cb467e5ca6bfc8e1a645b483bfd91b6b1648ced259110fc);

// examples/CRISP/.interfold/generated/contracts/ImageID.sol
bytes32 public constant PROGRAM_ID
= bytes32
  (0x7ed0ddeb0cafa64228b51399dff5ea00696c2617f28636b05bd67e6c1506eac3);
```

The deployment script explicitly acknowledges that the example copy can lag and reads the canonical file instead. It uses that value for both CRISP and the ciphertext verifier. The activation validator checks both configured image IDs against the canonical value.

The demonstrated consequence is inconsistent tracked artifacts. A developer or custom tool treating the example copy as the current release artifact could obtain an obsolete value, but no current CRISP deployment or verification path consuming that copy was identified.

Recommendations:

It is recommended to stop tracking the unused CRISP copy and ignore the generated artifact. If it must remain as a release mirror, synchronize it through an explicit artifact-export step and add an automated equality check against the canonical CRISP image ID.

Removing the duplicate or exporting an existing verified release artifact does not require rebuilding the guest. When regeneration is necessary, use the pinned Docker build and preserve the generated-artifact handling requirement in `agent/INVARIANTS.md`; do not manually edit the constant. Apply equality checks only to intentional release mirrors, since locally modified programs can legitimately have different image IDs.

Interfold: Resolved with [PR-1930](#) by removing the stale CRISP image ID copy from version control and ignoring the generated directory.

Zenith: Verified.

[I-7] Any account can write to another voter's input slot, with no on-chain sender or sibling-ordering binding

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target:

- [CRISPPProgram.sol#L589-L627](#)
- [CRISPPProgram.sol#L781-L787](#)
- [examples/CRISP/circuits/bin/crisp/src/main.nr#L208-L225](#)

Description:

`publishInput` never relates the caller to the slot it writes, so any account may submit an input for any `slotAddress`. The Noir circuit asserts the voter's signature and address only in the non-mask branch, so a mask ballot needs no signature from the slot owner, and in ONCHAIN census mode — the mode planned for the DAO — no census witness is needed either. `_parentCommitment` accepts any existing parent rather than the slot's current head:

```
function _parentCommitment(uint256 e3Id, address slotAddress,
    uint40 parentIndexPlusOne) internal view returns (bytes32) {
    if (parentIndexPlusOne == 0) return bytes32(0);
    bytes32 commitment
        = e3Data[e3Id].inputCommitment[slotAddress][parentIndexPlusOne - 1];
    if (commitment == bytes32(0)) revert UnknownParentInput(parentIndexPlusOne
        - 1);
    return commitment; // any existing parent passes; never required to be the
        slot's current head
}
```

Two submissions for the same voter can name the same parent and both pass the on-chain checks. These competing entries are siblings. The contract records their append order, but it does not require the caller to own the voter's slot or determine which entry becomes the slot's usable head.

The computation guest makes that selection. It follows each slot's parent chain and accepts the first usable entry that extends the current head. Once that entry becomes the new head, a later sibling naming the old parent is skipped. A voter's update can therefore be recorded successfully on-chain yet be skipped during computation if another usable

entry extends its parent first. The voter must then submit a new update using the selected head as its parent.

Recommendations:

Reject an append whose declared parent is not the slot's current head (`voteSlots[slotAddress]`) unless the submission explicitly opens a fresh slot (`parentIndexPlusOne = 0`), so a lost race reverts explicitly rather than resolving silently off-chain, while still permitting recovery when the head itself is unusable. Optionally bind the on-chain input to the slot owner. Keep the intentionally permissionless mask design.

Interfold: Acknowledged. the contract cannot verify the ciphertext bytes are valid (as we cannot compare it with the commitment onchain) so we need to be able to point ciphertexts to older heads otherwise one would be able to brick voting rounds by submitting an invalid ciphertext and all following votes would need to build from there. This way one can choose which ciphertext to bind to and it must be the last valid one.

[I-8] Committee key publication after the input window flips failure attribution to the requester

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [Interfold.sol#L807-L814](#)
- [../contracts/lib/InterfoldLifecycle.sol#L268-L277](#)

Description:

Nothing ties key publication to the input window:

```
function validateCommitteePublication(address caller, address registry,
    uint256 e3Id, uint8 current,
    uint256 dkgDeadline) external view {
    if (caller != registry) revert IInterfold.OnlyCiphernodeRegistry();
    if (IInterfold.E3Stage(current) != IInterfold.E3Stage.CommitteeFinalized)
        revert ...;
    > if (block.timestamp > dkgDeadline)
        revert IInterfold.DKGDeadlinePassed(e3Id, dkgDeadline);
    // no check that block.timestamp ≤ inputWindow[1]
}
```

A relayer with a valid DKG proof can publish just before dkgDeadline but after inputWindow[1] via `max(block.timestamp, inputWindow[1])`, reaching KeyPublished for a round that could not receive inputs, which then fails as a requester-paid ComputeTimeout instead of a nodes-paid DKGTimeout.

Recommendations:

Reject key publication after the input window closes, or attribute the resulting timeout to the committee.

Interfold: Resolved with [PR-1927](#).

Zenith: Verified.

[I-9] A committed input with no availability receipt bricks the round, billed to the requester

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [CRISPPProgram.sol#L619, #L643-L656, #L717-L726, #L937-L938](#)
- [../contracts/lib/FailurePayerLib.sol#L16-L27](#)

Description:

pendingInputCount is incremented by publishInput and decremented only by the deadline-gated finalizeInput; verify() refuses output while it is nonzero:

```
round.pendingInputCount++; // publishInput
round.pendingInputCount--; // finalizeInput, only reachable with a VectorX
                             receipt before the deadline
▷ if (pending ≠ 0) revert InputAvailabilityPending(pending); // verify()
                             blocks the whole round
```

A committed input remains pending until its availability receipt is verified. While any input is pending, the contract blocks ciphertext-output publication. If the receipt arrives after the compute deadline, the round can fail with ComputeTimeout.

The requester bears the timeout cost under FailurePayerLib.sol:16-23, including when external DA delays prevent an otherwise valid input from finalizing. This can happen to an honestly submitted input: admission requires both a valid ballot proof and the availability signer's attestation, but those checks do not guarantee timely DA finalization.

validateTimeoutConfig allows any positive computeWindow up to 30 days, without requiring a minimum duration for DA finality. A short window can therefore expire before an honest input's receipt becomes available.

Recommendations:

Add a bounded reconciliation of pendingInputCount once a committed input's finalization window has closed (permissionless per-input eviction after the deadline, or count only inputs still inside their window in verify()). Enforce a minimum computeWindow derived

from expected VectorX latency.

Interfold: Resolved with [PR-1930](#).

Zenith: Verified.

[I-10] An E3 program not implementing IE3ProgramDataAvailability bricks its own output publication

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [../contracts/lib/InterfoldLifecycle.sol#L371-L386](#)

Description:

_verifyDataAvailability casts the stored program to IE3ProgramDataAvailability and calls it unconditionally:

```
// InterfoldLifecycle._verifyDataAvailability
▷ receipt = IE3ProgramDataAvailability(e3Program).verifyDataAvailability
  (ciphertextOutputHash, availabilityProof);
// no code.length check, no supportsInterface probe, no try/catch
```

with no code.length check, supportsInterface probe, or try/catch. A program admitted by the trusted owner that omits the selector reverts on every output publication, bricking its own rounds.

Recommendations:

Check that the program implements the expected interface at registration, or wrap the call and surface a clear error.

Interfold: Resolved with [PR-1927](#). Both required interfaces checked.

Zenith: Verified.